TECNOLÓGICO NACIONAL DE MÉXICO

Inteligencia Artificial

Tarea:

Sistema de Enrutamiento

Alumno:

Rementeria Medina Jesus Hector

Análisis

La cadena de restaurantes en Culiacán depende de una red de centros de distribución para abastecer a sus sucursales con productos, este proceso enfrenta diversos desafíos logísticos que impactan directamente en la eficiencia operativa y en los costos de transporte, entre los principales problemas se encuentran la gran cantidad de puntos de entrega, la variabilidad en las distancias entre centros y tiendas, y la necesidad de cumplir con plazos de entrega estrictos

El tráfico urbano en Culiacán representa un factor adicional que puede retrasar las entregas y aumentar el consumo de combustible, las capacidades de almacenamiento y venta de cada tienda son diferentes, lo que obliga a planificar rutas que consideren tanto la demanda como la capacidad de recepción de cada sucursal. La falta de una estrategia optimizada puede generar recorridos más largos de lo necesario, incrementando los costos de combustible y reduciendo la eficiencia del sistema de distribución

En este contexto, surge la necesidad de modelar el problema como un caso de optimización combinatoria, la meta es encontrar rutas que minimicen la distancia total recorrida y el costo de combustible, garantizando al mismo tiempo que cada tienda reciba los productos en tiempo y forma, resolver este problema de manera exacta resulta complejo debido al número elevado de combinaciones posibles, por lo que se requiere el uso de algoritmos heurísticos que permitan obtener soluciones cercanas al óptimo en tiempos razonables de cómputo

Modelado

El problema de distribución de productos comestibles desde los centros de distribución hacia las sucursales puede representarse como un caso de optimización combinatoria, cada centro y cada tienda se consideran nodos dentro de una red, mientras que las distancias y los costos de combustible entre ellos constituyen los arcos que conectan dichos nodos, el objetivo es encontrar un conjunto de rutas que minimicen el costo total de transporte, considerando tanto la distancia recorrida como el consumo de combustible.

Para formalizar el problema, se define un grafo $G=(N,A)$, donde N es el conjunto de nodos y A es el conjunto de arcos que representan las posibles conexiones entre ellos. A cada arco (i,j) se le asigna un costo C_{ij}, calculado en función de la distancia y el consumo de combustible. La función objetivo consiste en minimizar la suma de los costos de todos los arcos recorridos en las rutas seleccionadas:

$$\text{Minimizar } Z = \sum_{i,j} C_{ij} \cdot x_{ij}$$

donde xij es una variable binaria que indica si la ruta entre el nodo i y el nodo j es utilizada, las restricciones principales del modelo incluyen:

-Cada tienda debe ser visitada al menos una vez para garantizar el abastecimiento.

-Las rutas deben iniciar y terminar en un centro de distribución.

-Se deben respetar las capacidades de almacenamiento y venta de cada tienda.

-El tiempo de entrega debe mantenerse dentro de límites razonables para evitar pérdidas por perecibilidad.

Este modelado convierte el problema en una variante del Vehicle Routing Problem (VRP), que es una extensión del clásico Traveling Salesman Problem (TSP). Debido a la complejidad combinatoria y al gran número de nodos involucrados, resolverlo mediante métodos exactos resulta impráctico. Por ello, se recurre a algoritmos heurísticos como el Recocido Simulado o la Búsqueda Tabú, que permiten explorar soluciones cercanas al óptimo en tiempos de cómputo razonables.

Selección de algoritmos

Para abordar el problema de optimización de rutas de distribución se consideraron dos algoritmos heurísticos: Recocido Simulado y Búsqueda Tabú, ambos son ampliamente utilizados en problemas de optimización combinatoria y presentan características que los hacen adecuados para el caso de estudio.

El Recocido Simulado se inspira en el proceso físico de enfriamiento de metales. Su principal ventaja es la capacidad de escapar de mínimos locales mediante la aceptación probabilística de soluciones peores en etapas iniciales, lo que permite explorar un espacio de soluciones más amplio, es especialmente útil en problemas con gran cantidad de nodos, como el presente, donde las combinaciones posibles de rutas son muy numerosas, además su implementación es relativamente sencilla y flexible, lo que facilita la adaptación a diferentes restricciones logísticas.

Por otro lado, la Búsqueda Tabú utiliza una memoria de soluciones previamente exploradas para evitar ciclos y repeticiones, este enfoque permite intensificar la búsqueda en áreas prometedoras del espacio de soluciones y es eficaz para problemas donde se requiere un refinamiento constante de las rutas, sin embargo su implementación suele ser más compleja, ya que requiere definir estructuras de

memoria y reglas de aspiración que controlen qué soluciones se consideran prohibidas y cuáles pueden aceptarse nuevamente.

Tras comparar ambos métodos, se determinó que el Recocido Simulado es el más adecuado para este proyecto. La razón principal es que el problema involucra una red extensa de centros de distribución y tiendas, lo que genera un espacio de soluciones muy amplio. En este contexto, la capacidad del Recocido Simulado para explorar soluciones diversas y escapar de mínimos locales resulta más valiosa que la intensificación de la Búsqueda Tabú, su simplicidad de implementación permite obtener resultados eficientes en tiempos de cómputo razonables, lo que lo convierte en una herramienta práctica para la optimización de rutas de distribución en Culiacán.

Implementación

```python
# Matriz de distancias
distancias = pd.read_excel( io: "matriz_distancias.xlsx", header=0).iloc[1:,1:].values

# Matriz de costos de combustible
costos = pd.read_excel( io: "matriz_costos_combustible.xlsx", header=0).iloc[1:,1:].values

def calcular_costo(ruta, matriz_distancias, matriz_costos):  2 usages
    costo_total = 0
    for i in range(len(ruta) - 1):
        origen = ruta[i]
        destino = ruta[i + 1]
        distancia = matriz_distancias[origen][destino]
        costo_combustible = matriz_costos[origen][destino]
        costo_total += distancia * costo_combustible
    return costo_total
```

La matriz distancias contiene los valores numéricos de las distancias entre cada par de nodos de la red de distribución, es decir, entre los centros de distribución y las sucursales

La matriz costos almacena los valores asociados al consumo de combustible para recorrer cada tramo entre dos nodos

Ambas matrices se cargan desde archivos Excel mediante la librería pandas, eliminando la primera fila y la primera columna para conservar únicamente los datos numéricos

La función calcular_costo se encarga de evaluar el costo total de una ruta determinada, esta función recibe como parámetros la lista de nodos que conforman la ruta, junto con las matrices distancias y costos, para cada par consecutivo de nodos en la ruta, se obtiene la distancia desde la matriz distancias y el costo de combustible desde la matriz costos, ambos valores se multiplican para calcular el costo parcial de ese tramo, y el resultado se acumula en la

variable costo_total, al finalizar el recorrido de la ruta, la función devuelve el costo total acumulado

```python
def generar_ruta_inicial(nodos):  1 usage
    ruta = list(range(nodos))
    random.shuffle(ruta)
    return ruta

def generar_vecino(ruta):  1 usage
    nueva_ruta = ruta.copy()
    i, j = random.sample(range(len(ruta)), k: 2)
    nueva_ruta[i], nueva_ruta[j] = nueva_ruta[j], nueva_ruta[i]
    return nueva_ruta
```

La función generar_ruta_inicial tiene como propósito crear una ruta de partida para el algoritmo heurístico, recibe como parámetro el número total de nodos y construye una lista con todos ellos en orden consecutivo, posteriormente, utiliza el método random.shuffle para desordenar la lista de manera aleatoria, generando así una ruta inicial distinta en cada ejecución, esta ruta sirve como punto de partida para el proceso de optimización

La función generar_vecino se encarga de producir una ruta alternativa a partir de una ruta existente, para ello, realiza una copia de la ruta actual y selecciona dos posiciones aleatorias dentro de la lista de nodos, luego intercambia los valores de esas posiciones, creando una nueva ruta que difiere ligeramente de la original, este procedimiento permite explorar el espacio de soluciones de manera controlada, ya que cada vecino representa una variación mínima de la ruta actual

En conjunto, generar_ruta_inicial proporciona la primera solución candidata, mientras que generar_vecino permite generar nuevas soluciones cercanas. Ambas funciones son esenciales para el funcionamiento del Recocido Simulado, ya que definen cómo se inicia la búsqueda y cómo se exploran las posibles rutas durante el proceso de optimización

```python
def recorido_simulado(matriz_distancias, matriz_costos, iteraciones=1000, temp_inicial=100, alfa=0.99):
    nodos = len(matriz_distancias)
    ruta_actual = generar_ruta_inicial(nodos)
    costo_actual = calcular_costo(ruta_actual, matriz_distancias, matriz_costos)

    mejor_ruta = ruta_actual
    mejor_costo = costo_actual

    temperatura = temp_inicial
    historial_costos = []

    for _ in range(iteraciones):
        vecino = generar_vecino(ruta_actual)
        costo_vecino = calcular_costo(vecino, matriz_distancias, matriz_costos)

        # Aceptación de vecino
        if costo_vecino < costo_actual:
            ruta_actual, costo_actual = vecino, costo_vecino
        else:
            prob = math.exp(-(costo_vecino - costo_actual) / temperatura)
            if random.random() < prob:
                ruta_actual, costo_actual = vecino, costo_vecino
```

```python
        # Actualizar mejor solución
        if costo_actual < mejor_costo:
            mejor_ruta, mejor_costo = ruta_actual, costo_actual

        historial_costos.append(mejor_costo)
        temperatura *= alfa  # enfriamiento

    return mejor_ruta, mejor_costo, historial_costos
```

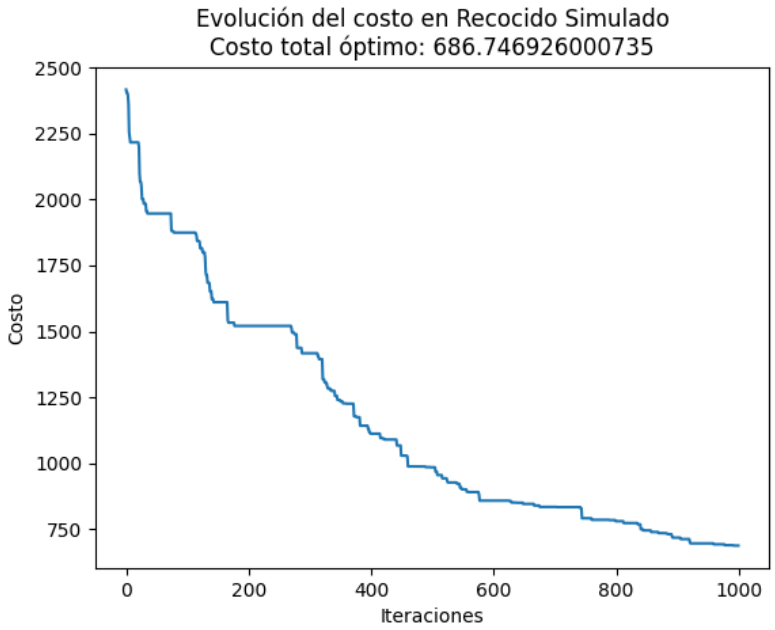
La función recorido_simulado implementa el algoritmo heurístico de Recocido Simulado para optimizar las rutas de distribución. Recibe como parámetros la matriz_distancias, la matriz_costos, el número de iteraciones, la temp_inicial (temperatura inicial) y el factor de enfriamiento alfa

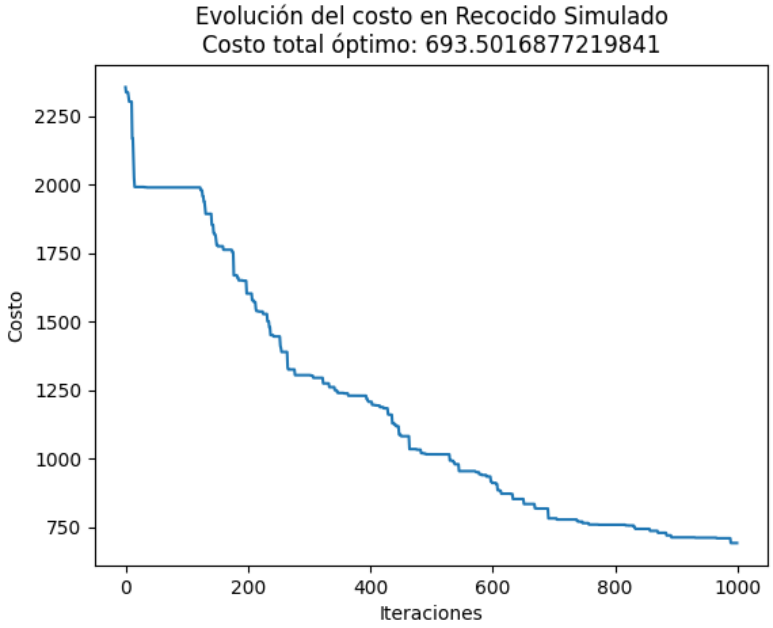
Se obtiene el número de nodos a partir de la matriz de distancias, se genera una ruta inicial aleatoria mediante la función generar_ruta_inicial y se calcula el costo de esa ruta con la función calcular_costo, se guarda esta ruta como la mejor solución inicial, se define la temperatura inicial y se crea una lista para almacenar el historial de costos, en cada iteración se genera un vecino de la ruta actual usando la función generar_vecino, se calcula el costo de este vecino

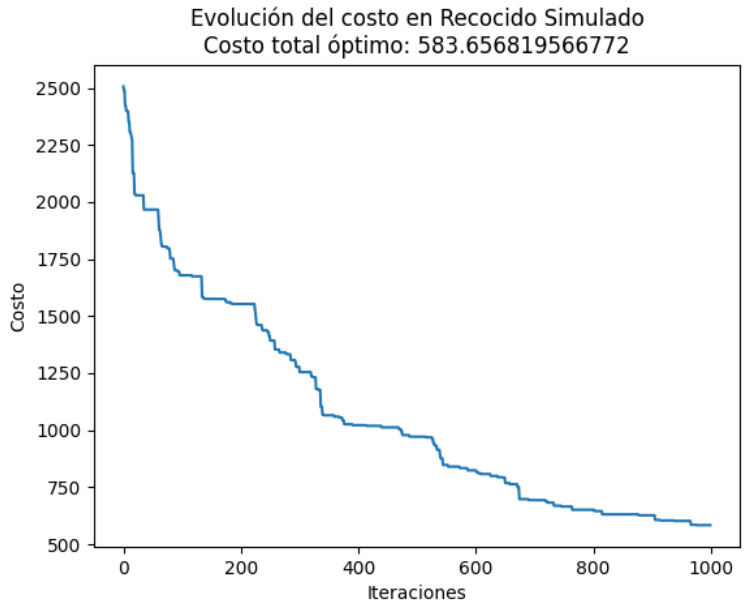
Si el vecino tiene menor costo que la ruta actual, se acepta directamente, si el vecino es peor, se acepta con una probabilidad que depende de la diferencia de costos y de la temperatura actual, esto permite escapar de mínimos locales

Se actualiza la mejor solución encontrada si el costo actual es menor que el mejor costo registrado, se guarda el mejor costo en el historial y se reduce la temperatura multiplicándola por el factor alfa y al terminar las iteraciones, la función devuelve la mejor ruta encontrada, su mejor costo y el historial de costos, que sirve para graficar la evolución del proceso de optimización.

Resultados






Evolución del costo en Recocido Simulado
Costo total óptimo: 583.656819566772
Costo
Iteraciones
2500
2250
2000
1750
1500
1250
1000
750
500
0
200
400
600
800
1000